

System Architecture

AI Investment Advisor · Capstone Project — Track 3

Guiding principle: the deterministic scoring engine decides; the LLM (GPT-4.1) only explains the result in plain language — it never overrides the rating.

1. Overview

The system follows a layered architecture with clear separation of concerns: an auth layer, a data layer, an analysis layer, a scoring engine, an AI layer (narration + chatbot), and a presentation layer (Streamlit), with app.py as a thin router that ties everything together.

Presentation — app.py (router) · src/ui/pages: landing · auth · dashboard
Auth — sign up / login (SQLAlchemy · PBKDF2) · signed session token
Orchestration — dashboard.py coordinates every layer below
Data — Finnhub (price/news) · yfinance (history) · DB (SQLite/Postgres)
Analysis — native indicators (RSI/MACD/EMA/Bollinger/StochRSI/ADX/ATR/OBV) + sentiment (FinBERT/VADER)
Scoring engine — Trend 35% (correlation-de-duplicated) · Momentum 25% · Volume 15% · Volatility 10% · Sentiment 10% → 0–100 score
AI — GPT-4.1 narrates the score (does not decide it)
Floating chatbot — Groq/Gemini, grounded (RAG-style) in the live analysis context only

2. Components

2.1 Auth layer

src/auth/models.py defines the SQLAlchemy tables: User (username, email, password hash) and SavedAnalysis (a user's analysis history). src/auth/service.py handles sign-up and login, hashes passwords with PBKDF2 (standard library only, no extra crypto dependency), and issues a signed session token stored in the URL query params so a login survives a page refresh.

2.2 Data layer

src/data/market_data.py pulls the current price from Finnhub and OHLCV history from yfinance (with Tiingo as an optional fallback). drop_incomplete_bar removes the still-forming candle to avoid look-ahead bias, and price_adjustment="raw" matches the convention used by TradingView, Investing.com and Barchart. src/data/news_data.py pulls headlines from the Finnhub News API. src/data/database.py uses SQLite for local development and Postgres in production via DATABASE_URL.

2.3 Analysis layer

src/analysis/indicators.py is a native NumPy/pandas implementation (not pandas-ta): Wilder's RMA for RSI/ATR/ADX, EMA for MACD, and population standard deviation for Bollinger Bands. It also covers EMA20/50/200, Stochastic RSI, OBV, Volume SMA, and swing-based support/resistance. src/analysis/sentiment.py runs FinBERT with a VADER fallback.

src/analysis/correlation.py automatically detects correlated trend indicators (Price>SMA50, Price>SMA200, SMA50>SMA200, EMA20>EMA50, and so on) and groups them into a cluster that shares a single weight, so the same phenomenon is never counted several times. It is wired directly into scoring.py's Trend category; ADX keeps its own fixed share (trend_adx_share) since it measures trend strength rather than direction, so it is deliberately kept out of the correlation cluster.

src/analysis/scoring.py is the deterministic scoring engine: it blends five active categories (Trend, Momentum, Volume, Volatility, Sentiment, weighted 35% / 25% / 15% / 10% / 10% in config.py, renormalised when data is missing) into a 0–100 score. config.py also defines weight_risk = 0.05, but there is no active Risk category in scoring.py — that weight is currently unused. Every indicator exposes a name, a detail string and a points value that feeds the category average, so nothing is a black box. Confidence is computed from a formula that blends distance from the neutral midpoint (the dominant term) with agreement across categories.

2.4 AI layer

src/ai/advisor.py takes the final rating, score, confidence and the full category breakdown, and produces a structured narrative (executive summary, bullish/bearish factors, risk assessment, short- and long-term outlook). It does not set the rating, only explains it, and is explicitly instructed not to change it. Without an OpenAI key, a deterministic fallback narrator builds the same fields directly from the score breakdown. src/ai/chatbot.py and src/ui/chatbot_widget.py implement a floating assistant that only receives the live context (price, indicators, news, recommendation) and is instructed to answer from it alone — a RAG-style discipline. Primary engine: Groq (Llama 3.3 70B); fallback: Gemini 2.0 Flash.

2.5 Presentation layer

app.py is a thin entry point: it restores the session from the token and routes between landing, login, register and dashboard. src/ui/pages/ holds landing.py, auth_view.py and dashboard.py (the main view: executive summary → market snapshot → indicators → interactive chart → news analysis → detailed AI analysis → saved history). src/ui/components.py renders TradingView Lightweight Charts. src/ui/theme.py holds the dark fintech CSS theme, and src/ui/nav.py is a small session-state router.

2.6 Configuration

config.py defines a frozen Settings dataclass: API keys (OpenAI, Finnhub, Groq, Gemini, Tiingo), DATABASE_URL, SECRET_KEY, and all the indicator and scoring-engine weight parameters.

3. End-to-end data flow

- Sign-in: the user registers or logs in (src/auth) → a signed session token is stored in the URL.
- Input: on the dashboard, the user enters a ticker.
- Market and news: market_data.get_quote (Finnhub) + get_history (yfinance, 2 years) + news_data.get_news (Finnhub News).
- Analysis: indicators.py computes every indicator; sentiment.py scores the headlines.
- Scoring: scoring.py produces a 0–100 score plus a confidence level.
- Narration: advisor.py (GPT-4.1) writes the rationale, risk summary and source links.
- Display: dashboard.py renders everything and saves the result to the DB (database.py).
- Chat: the user can ask the floating chatbot questions, grounded in that analysis context.

4. Key design decisions

Decision	Rationale
Deterministic scoring engine, LLM only narrates	Full reproducibility; the LLM can never bias or hallucinate the rating itself.
Native indicators (not pandas-ta)	Full control over the smoothing convention (Wilder), alignment with TradingView, and complete test coverage.
correlation.py written to de-duplicate signals	Wired into scoring.py's Trend category; ADX kept independent (trend_adx_share) since it measures strength, not direction.
Auth + DB (SQLAlchemy)	Turns the project from a one-off demo into an actual B2C product with per-user history.
Finnhub for price, yfinance for history	Finnhub is fast for a live quote; yfinance gives free, deep history.
TradingView Lightweight Charts	The charting library retail investors already recognise.
Streamlit Community Cloud, packaged with Docker	Deployed live and built directly from the main branch; the Dockerfile also lets the same image run on any container-based host (e.g. Hugging Face Spaces).

5. Error handling and resilience

- Every network call is wrapped in try/except with retries (history_retries).
- Missing data shows as “Data unavailable” in the UI — never a fabricated value.
- Without API keys, the scoring engine still runs in full; only the narrative and chatbot are disabled.

- tests/test_indicators.py proves there is no look-ahead bias; tests/test_scoring.py checks determinism; tests/test_validation.py sanity-checks real tickers.

6. Limitations and future work

Known limitations

- The 5% Risk Adjustment weight defined in config.py has no corresponding category implemented in scoring.py.
- Confidence still depends heavily on the score's distance from 50, not only on agreement between categories.
- Educational tool only; every indicator is backward-looking; a high score does not guarantee a return.
- News coverage skews English-language; the default SQLite setup isn't meant for high concurrent load.

Planned next

- Implement the Risk Adjustment category.
- Multi-stock portfolios, full historical backtesting, automated alerts, and additional data sources.